

LAPORAN RESMI
MEMBUAT BACKEND SOCIAL MEDIA DENGAN NODEJS DAN MONGODB



Disusun untuk Memenuhi Tugas
Mata Kuliah:
WORKSHOP APLIKASI BERBASIS WEB

Oleh:
JUNDIAH INDIRA MILKA TSAURI
3122521020

PROGRAM STUDI D3 TEKNIK INFORMATIKA
PSDKU LAMONGAN
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2023

Langkah-langkah:

1. Buat folder baru,
2. Lalu buka visual studio code, klik terminal dan jalankan perintah npm init

```
PS D:\PENS\smt_3\WorkshopAplikasiBerbasisWeb\191023> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (191023)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
About to write to D:\PENS\smt_3\WorkshopAplikasiBerbasisWeb\191023\package.json:

{
  "name": "191023",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}

Is this OK? (yes) yes
```

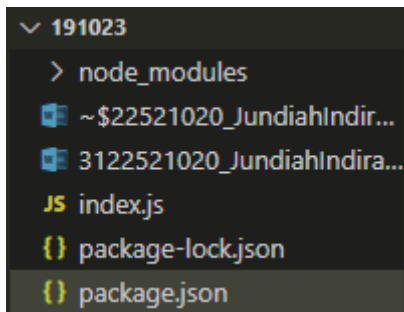
3. Jalankan perintah npm install express mongoose dotenv helmet morgan nodemon

```
PS D:\PENS\smt_3\WorkshopAplikasiBerbasisWeb\191023> npm install express mongoose dotenv helmet morgan nodemon
added 127 packages, and audited 128 packages in 15s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

4. Buat file baru dengan nama index.js



5. Tambahkan "start" pada package.js

```
"scripts": {  
  "start": "nodemon index.js",  
  "test": "echo \"Error: no test specified\" && exit 1"  
},
```

6. Isi index.js dengan:

```
const express = require("express");  
const app = express();  
app.listen(8800, () => {  
  console.log("Backend server is running!");  
});
```

Kemudian jalankan dengan perintah npm start

```
PS D:\PENS\smt_3\WorkshopAplikasiBerbasisWeb\191023> npm start  
  
> 191023@1.0.0 start  
> nodemon index.js  
  
[nodemon] 3.0.1  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting `node index.js`  
Backend server is running!
```

7. Buka cloud.mongodb.com, login atau daftar akun jika belum punya akunnya lalu ikuti langkah gambar dibawah ini:

Jundiah's Or...
Access Manager
Billing
All Clusters
Get Help
Jundiah

Project 0
Data Services
App Services
Charts

Overview
DEPLOYMENT
Database
Data Lake
SERVICES
Device Sync
Triggers
Data API
Data Federation
Search
Stream Processing
SECURITY
Backup
Database Access
Network Access
Advanced

JUNDIAH'S ORG - 2023-10-24 > PROJECT 0 > DATABASES
Cluster0
VERSION 6.0.11
REGION AWS N. Virginia (us-east-1)
CLUSTER TIER M0 Sandbox (General)

Overview
Real Time
Metrics
Collections
Search
Profiler
Performance Advisor
Online Archive
Cmd Line Tool

SANDBOX
NODES
REPLICA SET
Connect
Configuration
...

TAGS
Use tags to efficiently label and categorize your clusters. Any tags you apply will display here.
[Learn more about tagging.](#)
ADD TAG

REGION N. Virginia (us-east-1)

- ac-...shard-00-00.o... SECONDARY
- ac-...shard-00-01.o... PRIMARY
- ac-...shard-00-02.o... SECONDARY

This is a Shared Tier Cluster

If you need a database that's better for high-performance production applications, upgrade to a dedicated cluster.

Upgrade

Operations R: 0 W: 0
100.0/s
0
Last 6 Hours

Connect to Cluster0



Connect to your application



Drivers

Access your Atlas data using MongoDB's native drivers (e.g. Node.js, Go, etc.)



Access your data through tools



Compass

Explore, modify, and visualize your data with MongoDB's GUI



Shell

Quickly add & update data using MongoDB's Javascript command-line interface



MongoDB for VS Code

Work with your data in MongoDB directly from your VS Code environment



Connect to Cluster0



Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver	Version
Node.js	5.5 or later

2. Install your driver

Run the following on the command line

```
npm install mongodb
```

[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

☐ View full code sample

```
mongodb+srv://jundindira:<password>@cluster0.oqhltd.mongodb.net/?  
retryWrites=true&w=majority
```

Replace **<password>** with the password for the **jundindira** user. Ensure any option params are [URL encoded](#).

RESOURCES

[Get started with the Node.js Driver](#)
[Access your Database Users](#)

[Node.js Starter Sample App](#)
[Troubleshoot Connections](#)

8. Buat file baru dengan nama .env dan isi dengan code dari cluster0

```
.env  
1 MONGO_URL = mongodb+srv://jundindira:<password>@cluster0.oqhltd.mongodb.net/?retryWrites=true&w=majority
```

Isi username dan password sesuai dengan ketika daftarnya.

9. Buka index.js lalu tambahkan:

```
mongoose.connect(  
  process.env.MONGO_URL, {  
    useNewUrlParser: true, useUnifiedTopology: true },  
  )  
  .then (() => {  
    console.log("Connected to MongoDB");  
  })
```

```
.catch((error) => {  
    console.error("Error connecting to MongoDB:", error);  
});
```

Ketik perintah npm start

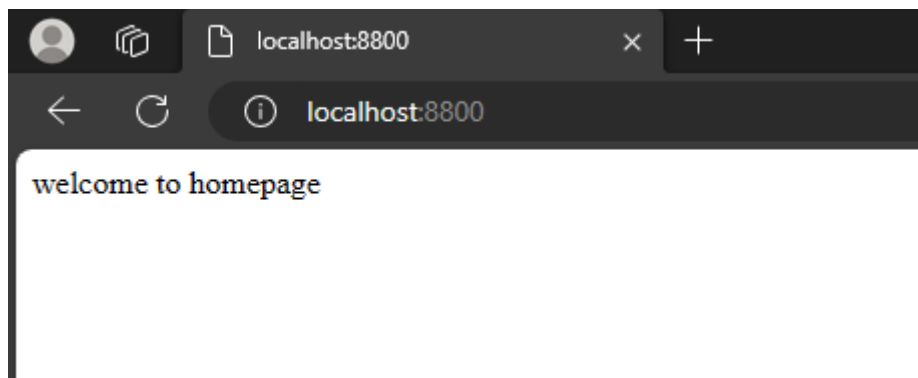
```
PS D:\PENS\smt_3\WorkshopAplikasiBerbasisWeb\191023> npm start  
  
> 191023@1.0.0 start  
> nodemon index.js  
  
[nodemon] 3.0.1  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting `node index.js`  
Backend server is running!  
Connected to MongoDB
```

10. Tambahkan koding

```
//middleware  
app.use(express.json());  
app.use(helmet());  
app.use(morgan("common"));  
  
app.get("/", (req,res)=>{  
    res.send("welcome to homepage")  
})
```

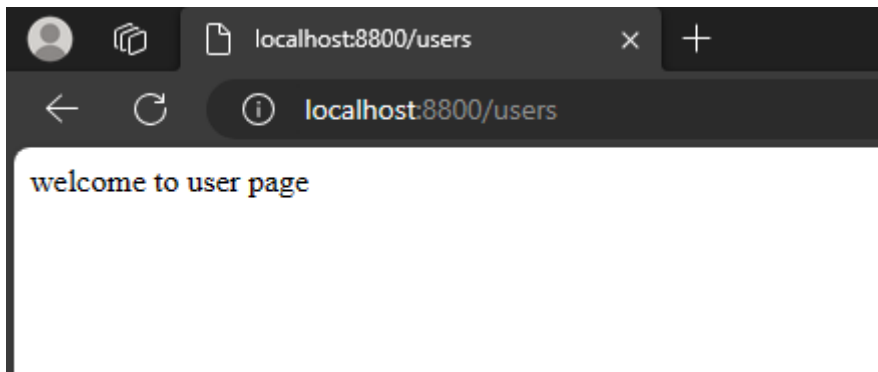
Dalam terminal akan otomatis menuliskan "Backend server is running!"

Buka browser,

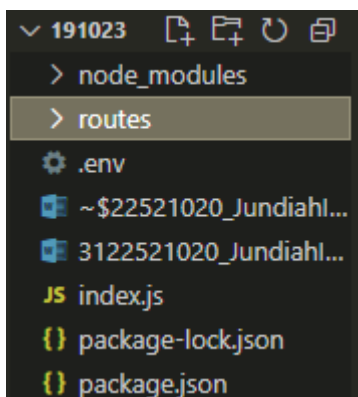


Jika ditambah koding:

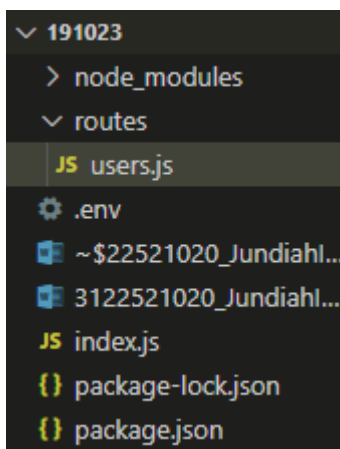
```
app.get("/users", (req,res)=>{  
    res.send("welcome to user page")  
})
```



11. Buat folder baru dengan nama routes



12. Buat file baru dalam routes



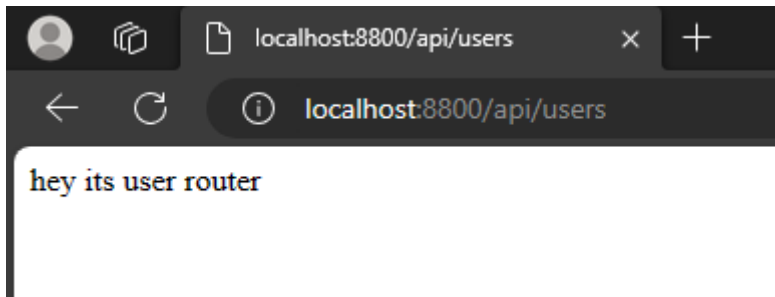
13. Buka file index.js, tambahkan koding

```
const userRoute = require("./routes/users");  
.  
.  
app.use("/api/users", userRoute);
```

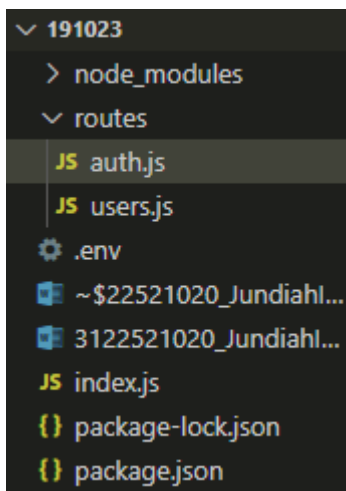
users.js diisi:

```
const router = require("express").Router();  
  
router.get("/", (req, res) => {  
  res.send("hey its user router")  
});
```

```
module.exports = router;
```



14. Buat file baru dalam routes dengan nama auth.js



Isi dengan koding:

```
const router = require("express").Router();

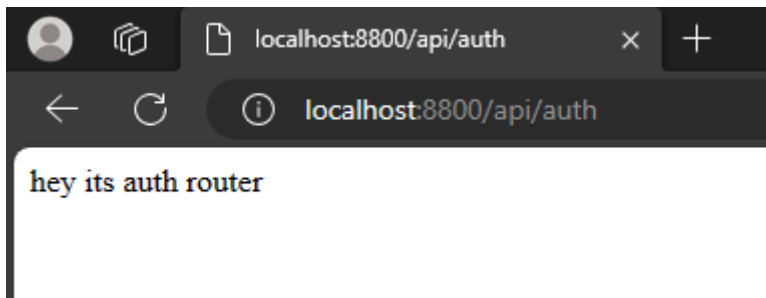
router.get("/", (req, res) => {
  res.send("hey its auth router")
});

module.exports = router;
```

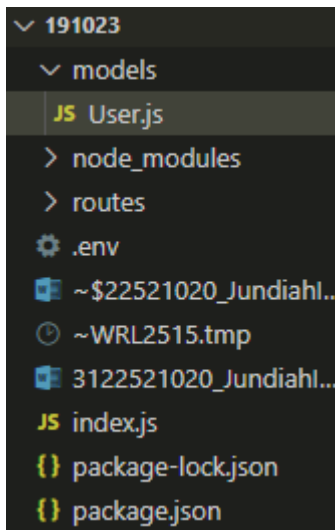
buka index.js, lalu tambahkan koding:

```
const authRoute = require("./routes/auth");
.
.
app.use("/api/auth", authRoute);
```

buka browser



15. Buat folder baru bernama models dan buat file bernama User.js



Isi dengan:

```
const mongoose = require("mongoose");

const UserSchema = new mongoose.Schema(
  {
    username: {
      type: String,
      required: true,
      min: 3,
      max: 20,
      unique: true,
    },
    email: {
      type: String,
      required: true,
      max: 50,
      unique: true,
    },
    password: {
      type: String,
      required: true,
      min: 6,
    },
    profilePicture: {
```

```

    type: String,
    default: "",
  },
  coverPicture: {
    type: String,
    default: "",
  },
  followers: {
    type: Array,
    default: [],
  },
  followings: {
    type: Array,
    default: [],
  },
  isAdmin: {
    type: Boolean,
    default: false,
  },
});

module.exports = mongoose.model("User", UserSchema);

```

16. Buka auth.js, tambah kodingan:

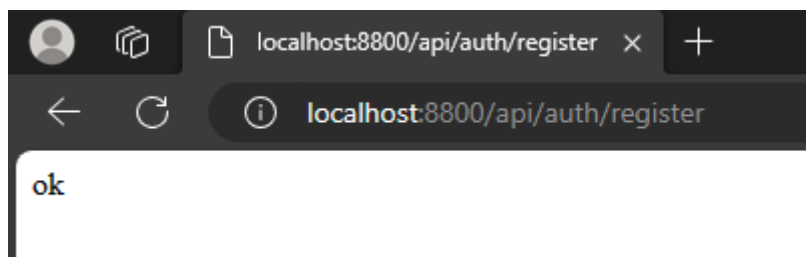
```

const User = require("../models/User");

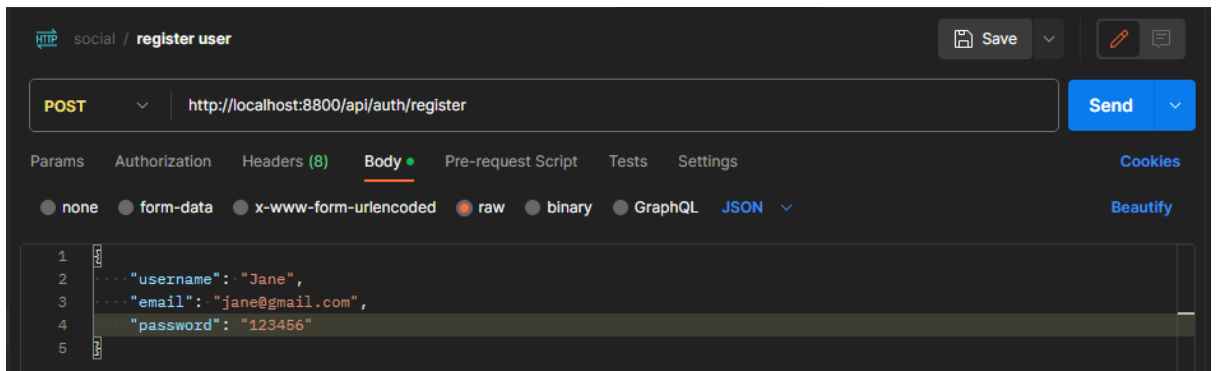
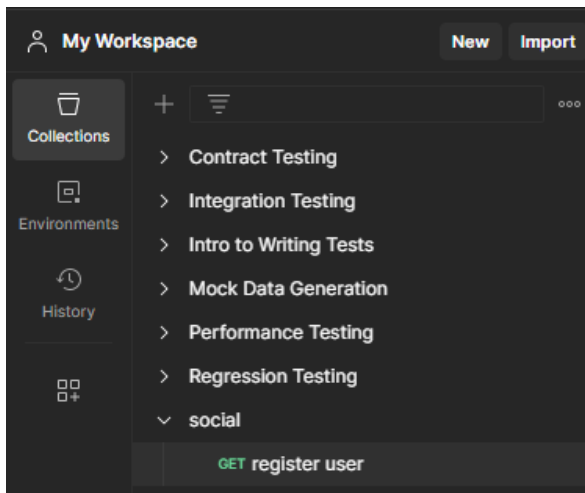
//REGISTER
router.get("/register", async (req, res) => {
  const user = await new User({
    username: "John",
    email: "john@gmail.com",
    password: "123456"
  })
  await user.save();
  res.send("ok")
});

```

Buka browser:



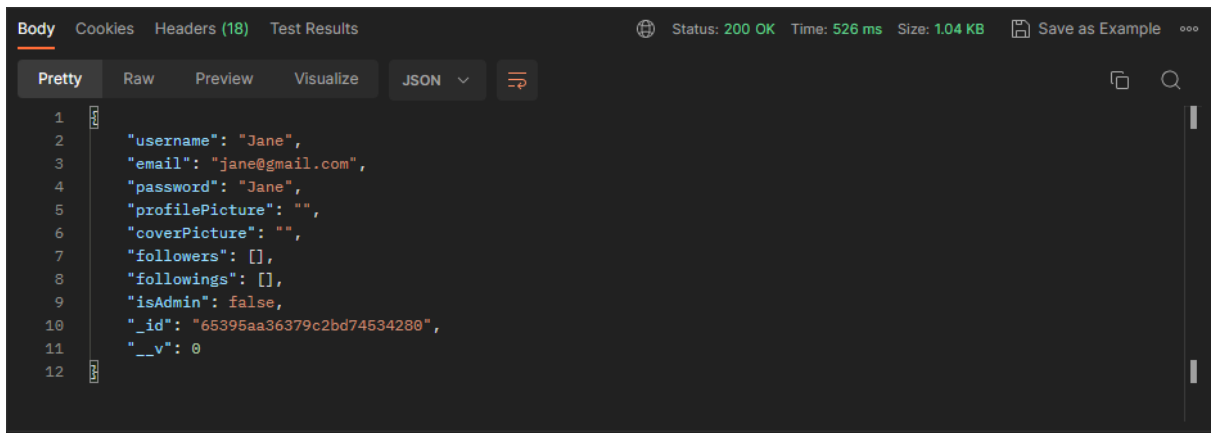
17. Buka Postman buat Collections baru



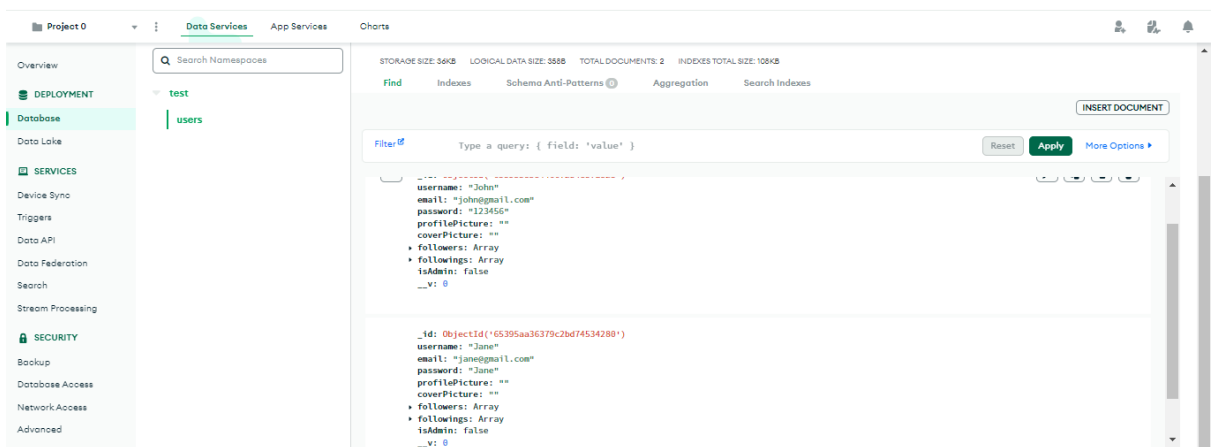
18. Buka auth.js. ada beberapa koding yang diubah

```
//REGISTER
router.post("/register", async (req, res) => {
  const newUser = new User({
    username: req.body.username,
    email: req.body.email,
    password: req.body.username,
  });
  try {
    const user = await newUser.save();
    res.status(200).json(user);
  } catch (err) {
    console.log(err);
  }
});
```

19. Buka Postman lagi lalu klik Send



20. Buka mongodb Atlas, klik Database lalu Collections, hasil Postman akan muncul disini



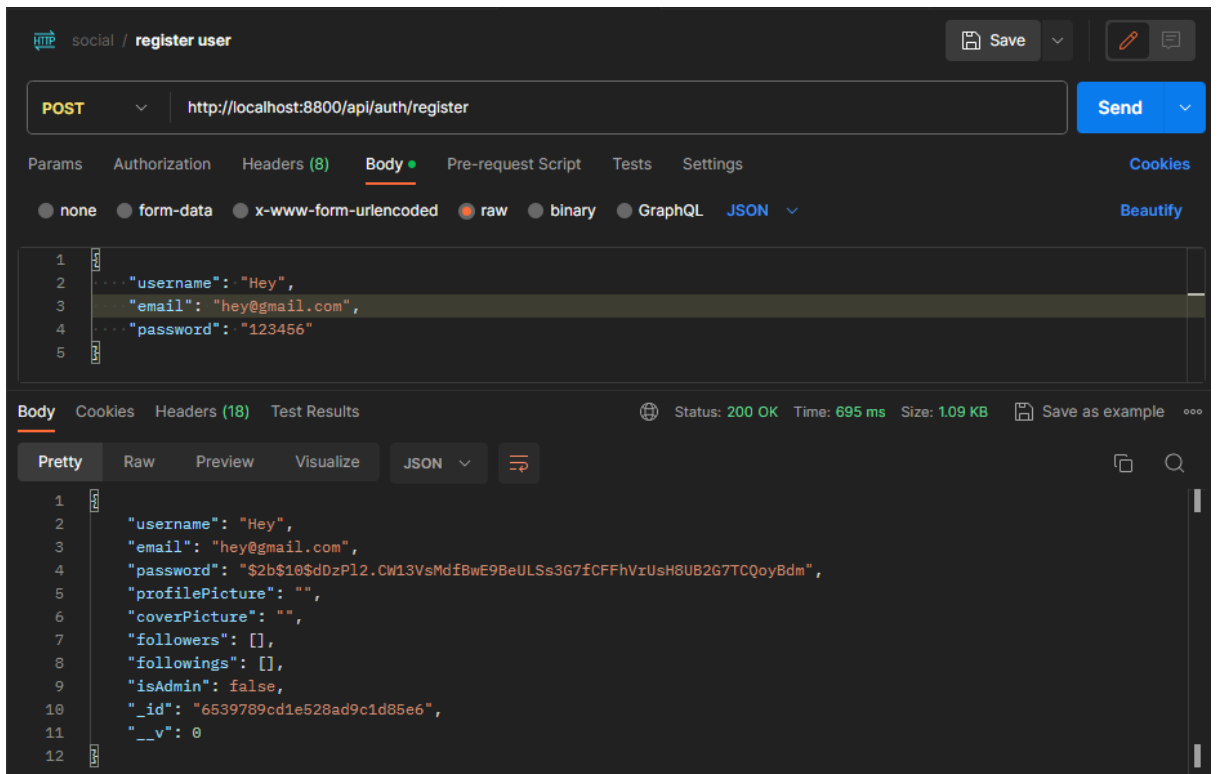
21. Jalankan perintah npm install bcrypt

```
PS D:\PENS\smt_3\WorkshopAplikasiBerbasisWeb\191023> npm install bcrypt
added 52 packages, and audited 180 packages in 7s

19 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

22. Buka postman lagi untuk menambahkan data:



23. Buka User.js, hasil semua kodingannya yaitu

```
const mongoose = require("mongoose");

const UserSchema = new mongoose.Schema(
  {
    username: {
      type: String,
      required: true,
      min: 3,
      max: 20,
      unique: true,
    },
    email: {
      type: String,
      required: true,
      max: 50,
      unique: true,
    },
    password: {
      type: String,
      required: true,
      min: 6,
    },
    profilePicture: {
      type: String,
      default: "",
    },
  },

```

```

    coverPicture: {
      type: String,
      default: "",
    },
    followers: {
      type: Array,
      default: [],
    },
    followings: {
      type: Array,
      default: [],
    },
    isAdmin: {
      type: Boolean,
      default: false,
    },
    desc: {
      type: String,
      max: 50,
    },
    city: {
      type: String,
      max: 50,
    },
    from: {
      type: String,
      max: 50,
    },
    relationship: {
      type: Number,
      enum: [1, 2, 3],
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", UserSchema);

```

24. Buka users.js dan tambahkan kodingan

```

//update user
router.put("/:id", async (req, res) => {
  if (req.body.userId === req.params.id || req.body.isAdmin) {
    if (req.body.password) {
      try {
        const salt = await bcrypt.genSalt(10);

```

```

        req.body.password = await bcrypt.hash(req.body.password,
salt);
    } catch (err) {
        return res.status(500).json(err);
    }
}
try {
    const user = await User.findByIdAndUpdate(req.params.id, {
        $set: req.body,
    });
    res.status(200).json("Account has been updated");
} catch (err) {
    return res.status(500).json(err);
}
} else {
    return res.status(403).json("You can update only your account!");
}
});

```

Run kembali

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8800/api/auth/register`
- Method:** `POST`
- Body (raw):**

```

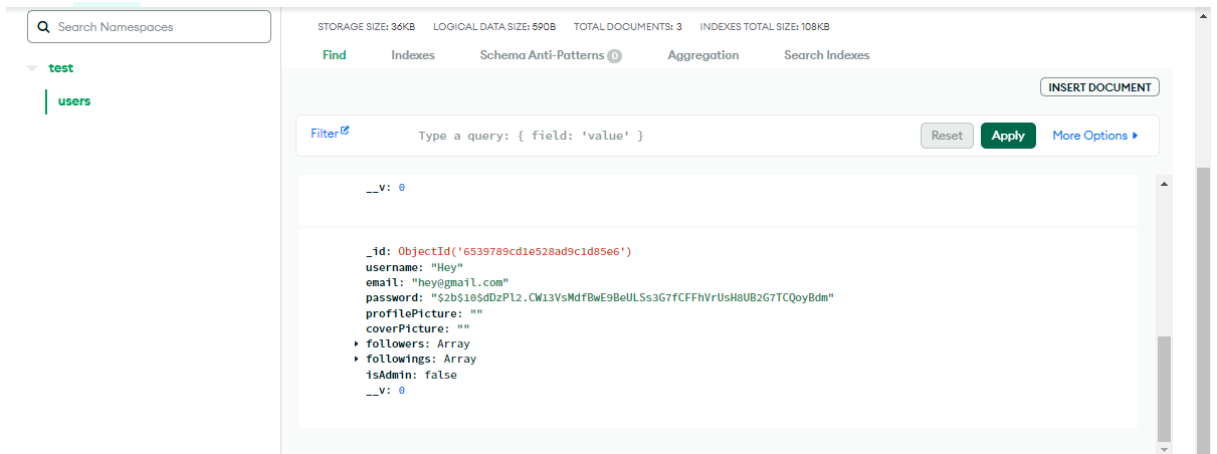
{
  "username": "Hey",
  "email": "hey@gmail.com",
  "password": "123456"
}

```
- Status:** `500 Internal Server Error`
- Time:** `391 ms`
- Size:** `976 B`
- Body (JSON):**

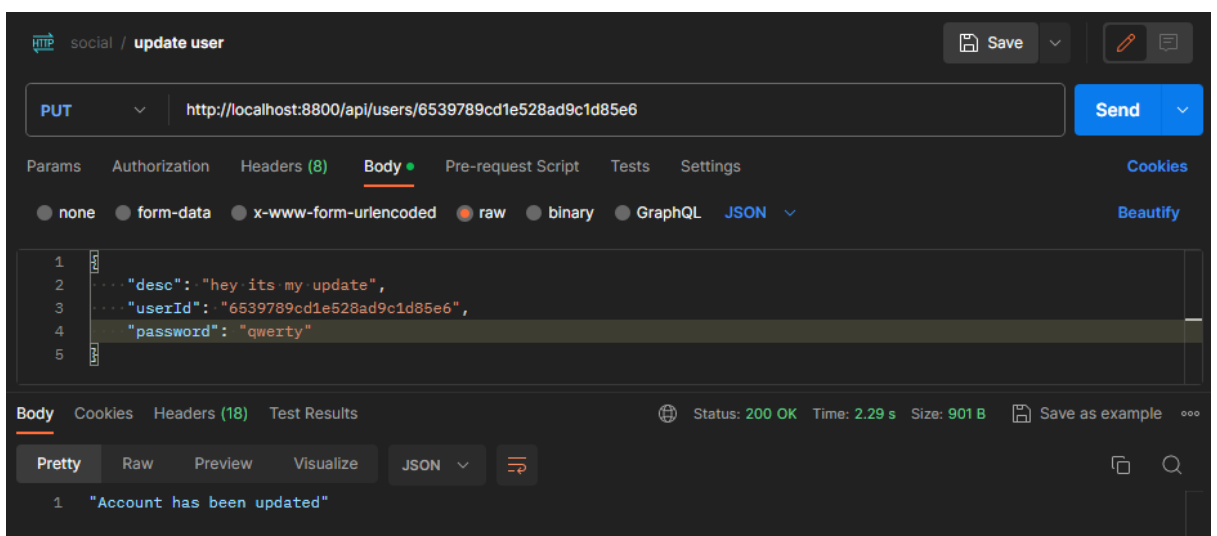
```

{
  "index": 0,
  "code": 11000,
  "keyPattern": {
    "username": 1
  },
  "keyValue": {
    "username": "Hey"
  }
}

```



25. Buka Postman, buat add request dengan nama update user



```

_id: ObjectId('6539789cd1e528ad9c1d85e6')
username: "Hey"
email: "hey@gmail.com"
password: "$2b$10$9ePxwAHbAdt91AE1QtrzT0QRZ706bchig7N9jHYL1UZGIQf07JWQm"
profilePicture: ""
coverPicture: ""
followers: Array
followings: Array
isAdmin: false
__v: 0
desc: "hey its my update"
updatedAt: 2023-10-25T22:35:51.477+00:00

```

26. Di user.js tambahkan koding

```

//delete user
router.delete("/:id", async (req, res) => {
  if (req.body.userId === req.params.id || req.body.isAdmin) {
    try {
      await User.findByIdAndDelete(req.params.id);
      res.status(200).json("Account has been deleted");
    } catch (err) {
      return res.status(500).json(err);
    }
  } else {

```

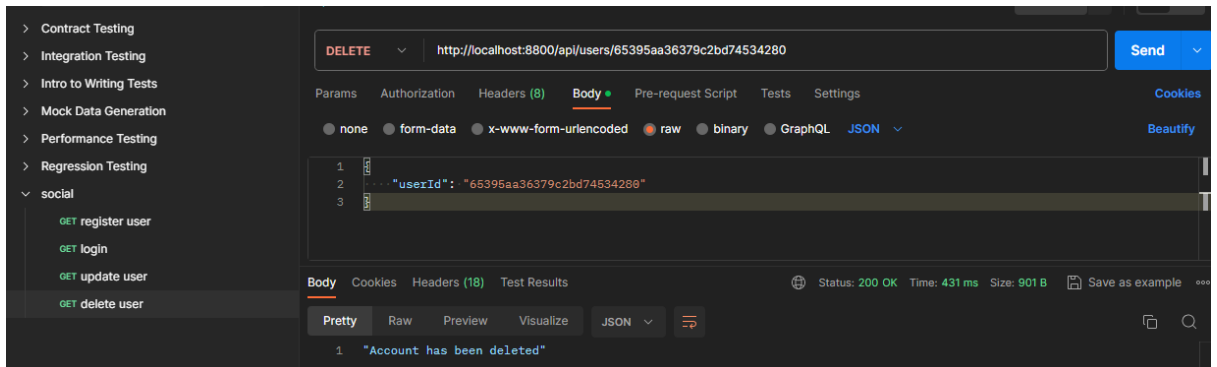


```

    return res.status(403).json("You can delete only your account!");
  }
});

```

27. Buat request dengan nama delete user



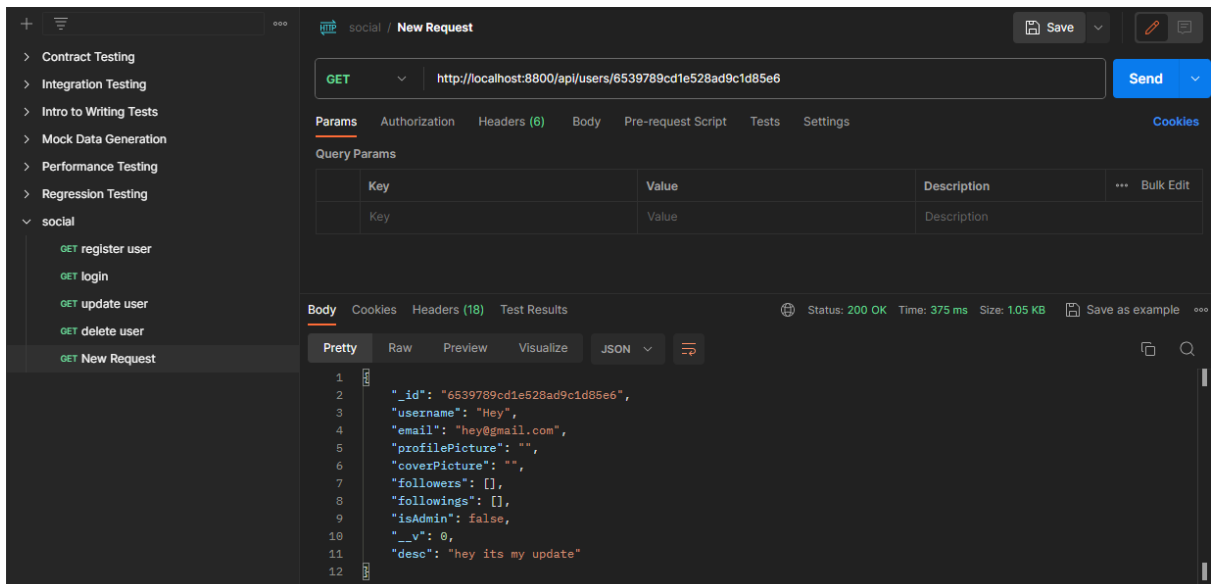
28. Tambahkan user.js dengan kodingan

```

//get a user
router.get("/:id", async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    const { password, updatedAt, ...other } = user._doc;
    res.status(200).json(other);
  } catch (err) {
    res.status(500).json(err);
  }
});

```

29. Buat request baru di Postman



30. User.js tambah kodingan:

```

//follow a user
router.put("/:id/follow", async (req, res) => {
  if (req.body.userId !== req.params.id) {

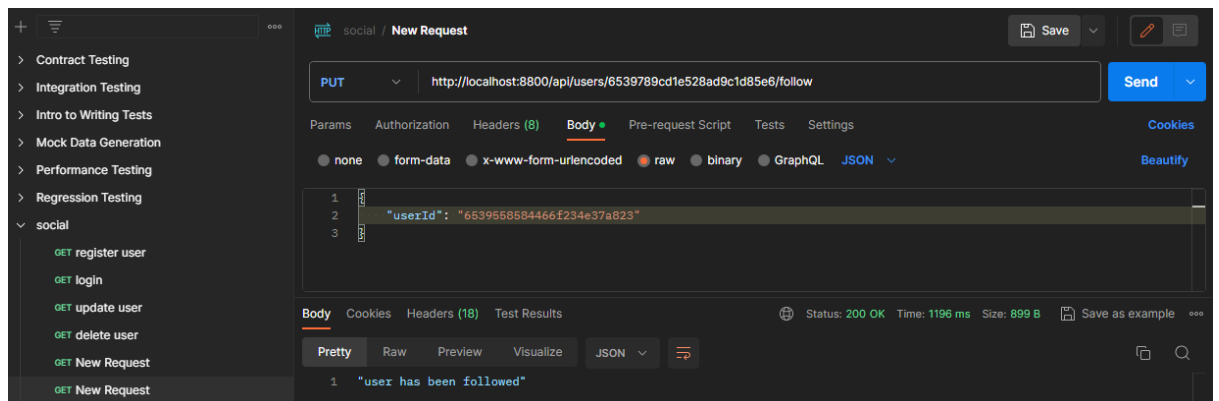
```

```

    try {
      const user = await User.findById(req.params.id);
      const currentUser = await User.findById(req.body.userId);
      if (!user.followers.includes(req.body.userId)) {
        await user.updateOne({ $push: { followers: req.body.userId }
      });
      await currentUser.updateOne({ $push: { followings:
req.params.id } });
      res.status(200).json("user has been followed");
    } else {
      res.status(403).json("you already follow this user");
    }
  } catch (err) {
    res.status(500).json(err);
  }
} else {
  res.status(403).json("you cant follow yourself");
}
});

```

31. Buat request baru



```

{
  "_id": ObjectId("6539558584466f234e37a823"),
  "username": "John",
  "email": "john@gmail.com",
  "password": "123456",
  "profilePicture": "",
  "coverPicture": "",
  "followers": Array,
  "followings": Array,
  "isAdmin": false,
  "_v": 0,
  "updatedAt": 2023-10-26T00:26:51.412+00:00
}

```

```

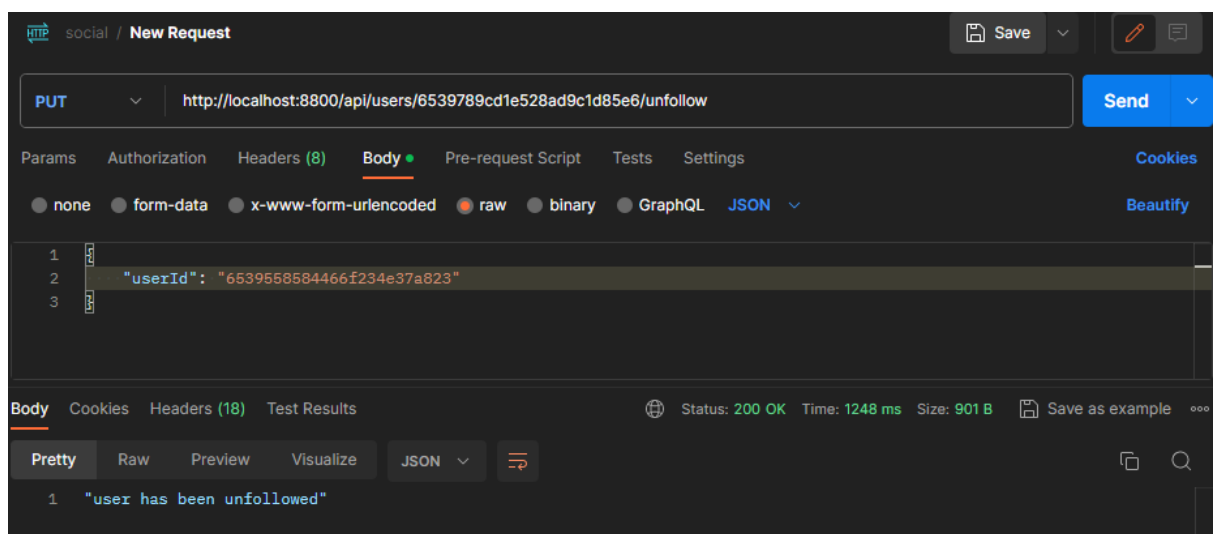
{
  "_id": ObjectId("6539789cd1e528ad9c1d85e6"),
  "username": "Hey",
  "email": "hey@gmail.com",
  "password": "$2b$10$9ePxAHbAdt9iAE1QtrzTQQRZ706bchig7N9jHYL1UzGIQf07JWQm",
  "profilePicture": "",
  "coverPicture": "",
  "followers": Array,
  "followings": Array,
  "isAdmin": false,
  "_v": 0,
  "desc": "hey its my update",
  "updatedAt": 2023-10-26T00:26:51.091+00:00
}

```

32. Tambah kodingan di user.js

```
//unfollow a user
router.put("/:id/unfollow", async (req, res) => {
  if (req.body.userId !== req.params.id) {
    try {
      const user = await User.findById(req.params.id);
      const currentUser = await User.findById(req.body.userId);
      if (user.followers.includes(req.body.userId)) {
        await user.updateOne({ $pull: { followers: req.body.userId } });
        await currentUser.updateOne({ $pull: { followings: req.params.id } });
        res.status(200).json("user has been unfollowed");
      } else {
        res.status(403).json("you dont follow this user");
      }
    } catch (err) {
      res.status(500).json(err);
    }
  } else {
    res.status(403).json("you cant unfollow yourself");
  }
});
```

33. Buka Postman,



```
_id: ObjectId('6539558584466f234e37a823')
username: "John"
email: "john@gmail.com"
password: "123456"
profilePicture: ""
coverPicture: ""
followers: Array
followings: Array
isAdmin: false
__v: 0
updatedAt: 2023-10-26T01:01:28.875+00:00
```

```
_id: ObjectId('6539789cd1e528ad9c1d85e6')
username: "Hey"
email: "hey@gmail.com"
password: "$2b$10$9ePwwAHbAdt9iAE1QtrzTQQRZ70Gbchig7W9jHYL1UZGIQf07JWQm"
profilePicture: ""
coverPicture: ""
followers: Array
followings: Array
isAdmin: false
__v: 0
desc: "hey its my update"
updatedAt: 2023-10-26T01:01:28.596+00:00
```

34. Buat file baru di folder models dengan nama Post.js

```
const mongoose = require("mongoose");

const PostSchema = new mongoose.Schema(
  {
    userId: {
      type: String,
      required: true,
    },
    desc: {
      type: String,
      max: 500,
    },
    img: {
      type: String,
    },
    likes: {
      type: Array,
      default: [],
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("Post", PostSchema);
```

35. Buat file baru bernama posts.js di folder routes

```
const router = require("express").Router();
const Post = require("../models/Post");
const User = require("../models/User");

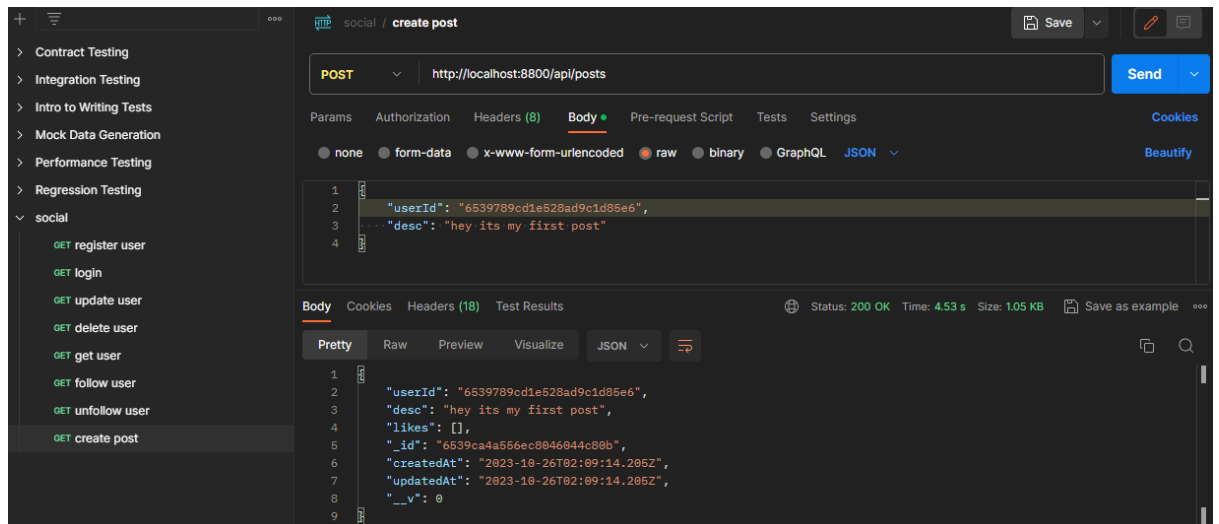
//create a post
```

```

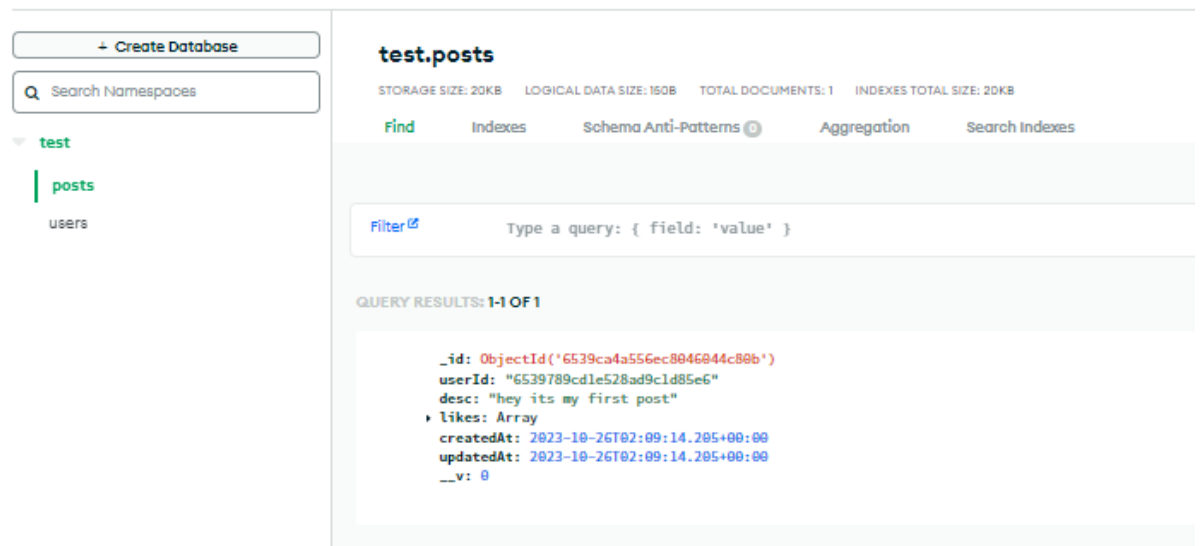
router.post("/", async (req, res) => {
  const newPost = new Post(req.body);
  try {
    const savedPost = await newPost.save();
    res.status(200).json(savedPost);
  } catch (err) {
    res.status(500).json(err);
  }
});

```

36. Buat request baru bernama create post



DATABASES: 1 COLLECTIONS: 2



social / create post

POST http://localhost:8800/api/posts

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```

1  {
2    "userId": "6539789cd1e528ad9c1d85e6",
3    "desc": "hey its my first post from hey",
4    "img": "image.png",
5  }

```

Body Cookies Headers (18) Test Results Status: 200 OK Time: 10.30 s Size: 1.08 KB Save as example

Pretty Raw Preview Visualize JSON

```

1  {
2    "userId": "6539789cd1e528ad9c1d85e6",
3    "desc": "hey its my first post from hey",
4    "img": "image.png",
5    "likes": [],
6    "_id": "6539cbb1556ec8046044c80d",
7    "createdAt": "2023-10-26T02:15:13.678Z",
8    "updatedAt": "2023-10-26T02:15:13.678Z",
9    "__v": 0
10 }

```

test.posts

STORAGE SIZE: 36KB LOGICAL DATA SIZE: 328B TOTAL DOCUMENTS: 2 INDEXES TOTAL SIZE: 36KB

Find Indexes Schema Anti-Patterns Aggregation Search Indexes

Filter Type a query: { field: 'value' }

QUERY RESULTS: 1-2 OF 2

```

_id: ObjectId('6539ca4a556ec8046044c80b')
userId: "6539789cd1e528ad9c1d85e6"
desc: "hey its my first post"
likes: Array
createdAt: 2023-10-26T02:09:14.205+00:00
updatedAt: 2023-10-26T02:09:14.205+00:00
__v: 0

```

```

_id: ObjectId('6539cbb1556ec8046044c80d')
userId: "6539789cd1e528ad9c1d85e6"
desc: "hey its my first post from hey"
img: "image.png"
likes: Array
createdAt: 2023-10-26T02:15:13.678+00:00
updatedAt: 2023-10-26T02:15:13.678+00:00
__v: 0

```

37. Update post dalam posts.js

```

//update a post
router.put("/:id", async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (post.userId === req.body.userId) {
      await post.updateOne({ $set: req.body });
      res.status(200).json("the post has been updated");
    } else {
      res.status(403).json("you can update only your post");
    }
  }
});

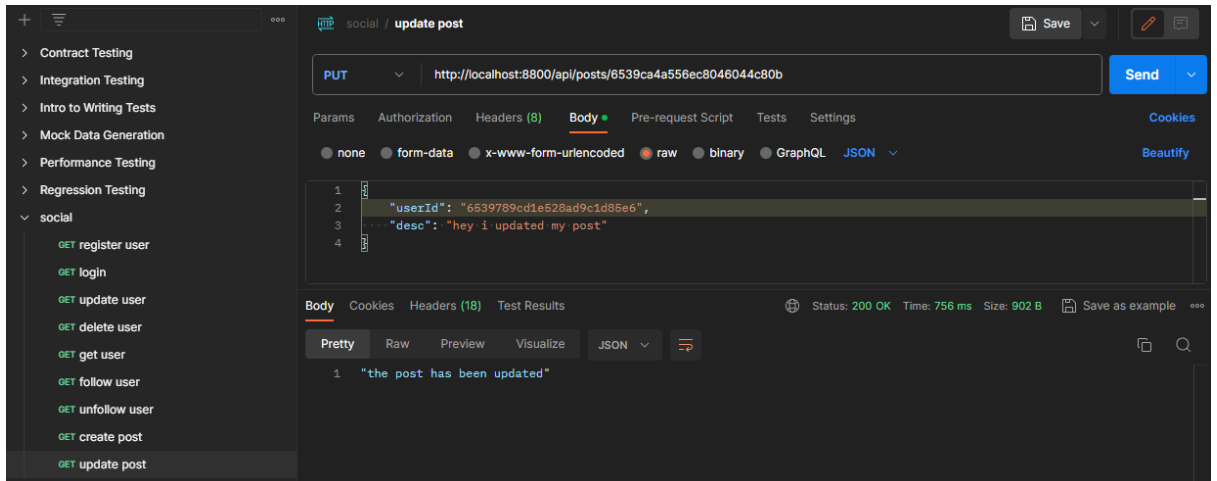
```

```

    }
  } catch (err) {
    res.status(500).json(err);
  }
});

```

38. Buat request update post



```

_id: ObjectId('6539ca4a556ec8046044c80b')
userId: "6539789cd1e528ad9c1d85e6"
desc: "hey i updated my post"
likes: Array
createdAt: 2023-10-26T02:09:14.205+00:00
updatedAt: 2023-10-26T02:32:30.870+00:00
__v: 0

```

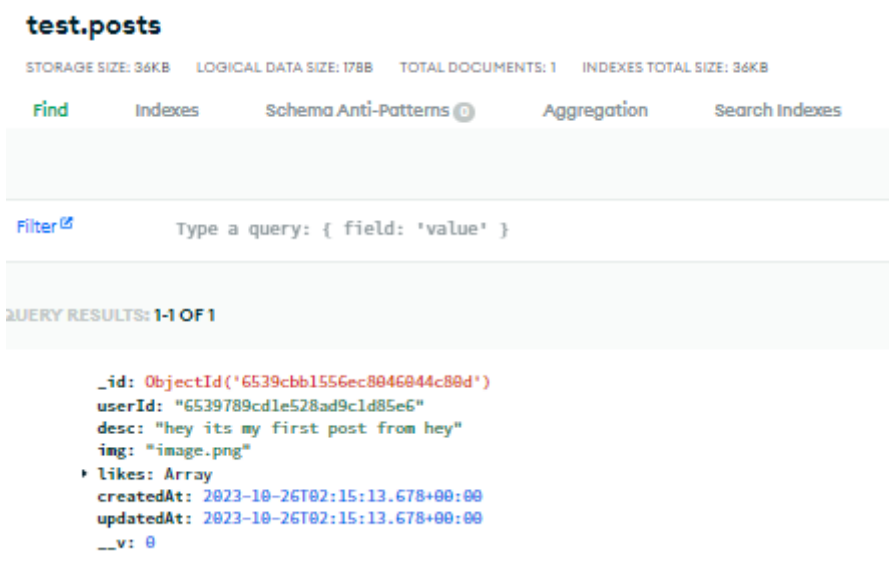
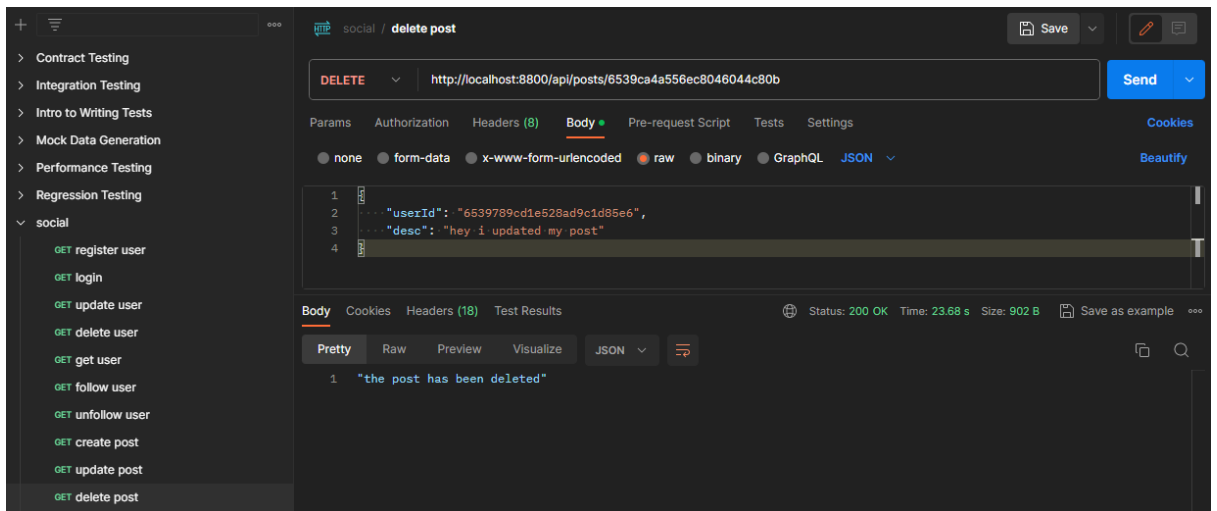
39. Tambah kodingan user.js

```

//delete a post
router.delete("/:id", async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (post.userId === req.body.userId) {
      await post.deleteOne();
      res.status(200).json("the post has been deleted");
    } else {
      res.status(403).json("you can delete only your post");
    }
  } catch (err) {
    res.status(500).json(err);
  }
});

```

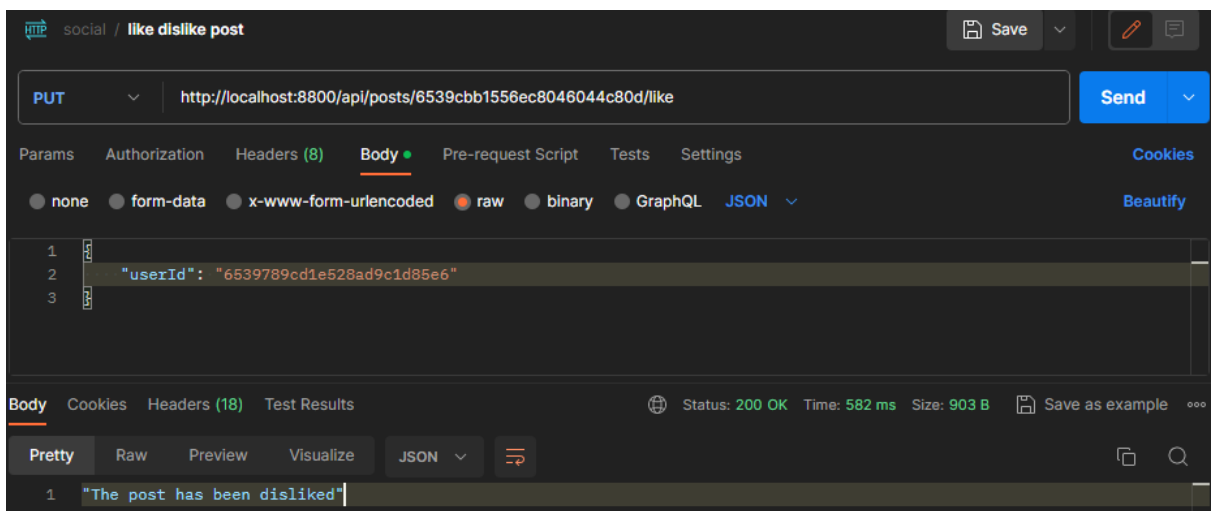
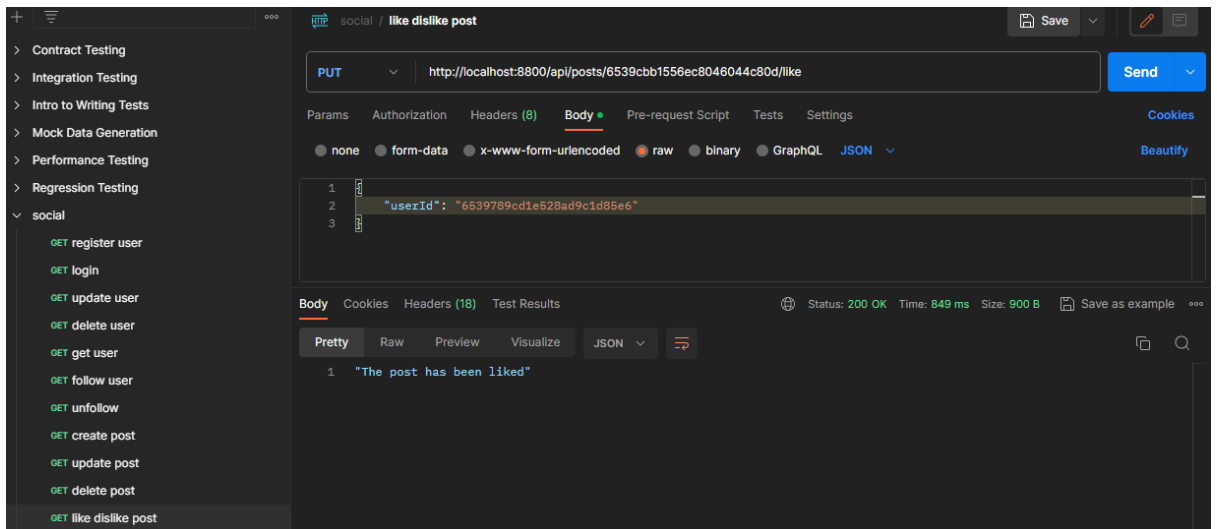
40. Buat request delete post



41. Tambah koding untuk like atau dislike post

```
//like / dislike a post
router.put("/:id/like", async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (!post.likes.includes(req.body.userId)) {
      await post.updateOne({ $push: { likes: req.body.userId } });
      res.status(200).json("The post has been liked");
    } else {
      await post.updateOne({ $pull: { likes: req.body.userId } });
      res.status(200).json("The post has been disliked");
    }
  } catch (err) {
    res.status(500).json(err);
  }
});
```

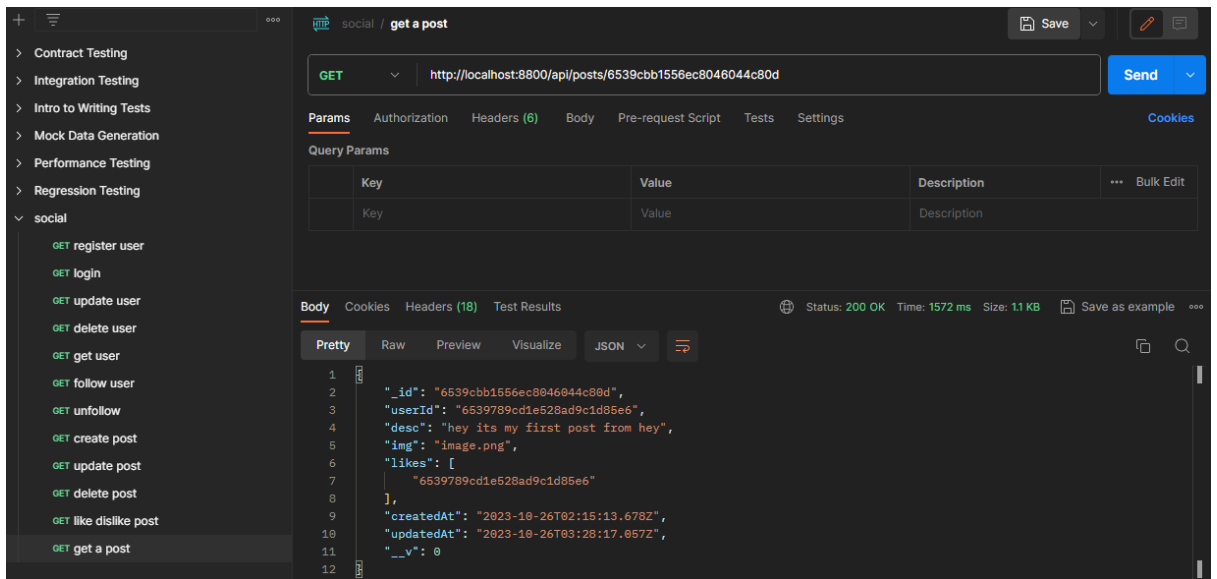
42. Buat request like dislike post



43. Tambah kodingan user.js

```
//get a post
router.get("/:id", async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    res.status(200).json(post);
  } catch (err) {
    res.status(500).json(err);
  }
});
```

44. Buat request get a post di postman



45. Tambahkan kodingan untuk timeline post

```

//get timeline posts
router.get("/timeline/all", async (req, res) => {
  try {
    const currentUser = await User.findById(req.body.userId);
    const userPosts = await Post.find({ userId: currentUser._id });
    const friendPosts = await Promise.all(
      currentUser.followings.map((friendId) => {
        return Post.find({ userId: friendId });
      })
    );
    res.json(userPosts.concat(...friendPosts))
  } catch (err) {
    res.status(500).json(err);
  }
});

```

46. Buat request di Postman

social / timeline post

GET http://localhost:8800/api/posts/timeline

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1
2 "userId": "6539789cd1e528ad9c1d85e6"
3
```

Body Cookies Headers (18) Test Results

Status: 500 Internal Server Error Time: 28 ms Size: 1.11 KB Save as example

Pretty Raw Preview Visualize JSON

```
1
2 "stringValue": "\"timeline\"",
3
4 "kind": "ObjectId",
5 "value": "timeline",
6 "path": "_id",
7 "reason": {},
8 "name": "CastError",
9 "message": "Cast to ObjectId failed for value \"timeline\" (type string) at path \"_id\" for model \"Post\""
10
```

social / timeline post

GET http://localhost:8800/api/posts/timeline/all

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1
2 "userId": "6539789cd1e528ad9c1d85e6"
3
```

Body Cookies Headers (18) Test Results

Status: 200 OK Time: 5.01 s Size: 1.1 KB Save as example

Pretty Raw Preview Visualize JSON

```
1
2 {
3   "_id": "6539cbb1556ec8046044c80d",
4   "userId": "6539789cd1e528ad9c1d85e6",
5   "desc": "hey its my first post from hey",
6   "img": "image.png",
7   "likes": [
8     "6539789cd1e528ad9c1d85e6"
9   ],
10  "createdAt": "2023-10-26T02:15:13.678Z",
11  "updatedAt": "2023-10-26T03:28:17.057Z",
12  "__v": 0
13 }
14
```